

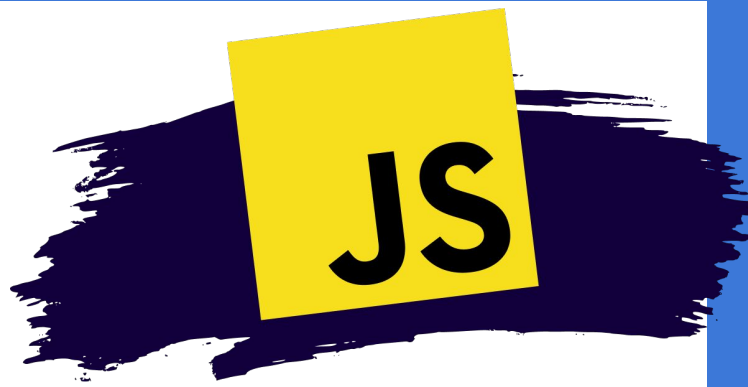


Javascript avanzado

Módulo 3 - BOOTCAMP Full Stack Web Development

Tema 9

Introducción a Typescript.



¿Qué es Typescript?

“TypeScript es un lenguaje de programación fuertemente tipado que se basa en JavaScript y le brinda mejores herramientas a cualquier escala.”

– definición de la [web oficial](#) -

Más concretamente, Typescript es un lenguaje de programación de código abierto desarrollado por Microsoft y un ‘super-set’ de Javascript, es decir, incorpora todo lo que nos ofrece Javascript y además una serie de funcionalidades extra como por ejemplo el tipado estático, las guards o los decoradores. La principal ventaja de utilizar Typescript es la detección temprana de errores durante el desarrollo a través de la transpilación a Javascript.



¿Qué es Transpilar?

Compilar es generar código ejecutable por una máquina, que puede ser física o abstracta como la máquina virtual de Java.

Transpilar es generar a partir de código en un lenguaje código en otro lenguaje. Es decir, un programa produce otro programa en otro lenguaje cuyo comportamiento es el mismo que el original.

Tipado estático

El [tipado estático](#) de TypeScript permite detectar errores relacionados con el tipo en tiempo de transpilación en lugar de en tiempo de ejecución . Esto puede ayudar a escribir código más confiable y detectar errores en las primeras etapas del proceso de desarrollo. TypeScript permite especificar los tipos de variables, parámetros de funciones y valores de retorno.

```
let message: string = 'Hello world';  
  
message = 3; // Error: variable of type 'string' is not assignable to type 'number'  
  
message(); // This expression is not callable. Type 'String' has no call signatures.
```

Tipos genéricos

Al ser un super-set de Javascript, Typescript va a contar con los tipos primitivos, arrays, objetos, etc. que hemos visto anteriormente. Además vamos a tener una serie de tipos adicionales:

- **void**: indica la ausencia de datos o 'vacío'. Podemos usarlo para funciones que no devuelven ningún valor.
- **any**: utilizado como tipo comodín, se le puede asignar un valor de cualquier tipo. Solo debemos usarlo en fases iniciales de desarrollo.
- **object**: similar a any pero restringido a objetos.

Interfaces

Las [interfaces](#) definen la estructura de un objeto , pero no proporcionan una implementación. Son una forma de describir la forma esperada de un objeto , lo que puede ayudar a garantizar que su código funcione como se espera. Las interfaces definen la forma de un objeto y pueden usarse para garantizar que un objeto se ajuste a una determinada estructura.

```
interface Person {  
  name: string;  
  age: number;  
}
```

```
function greet(person: Person) {  
  return "Hello " + person.name;  
}
```

Inferencia de tipos

TypeScript puede inferir el tipo de una variable en función de su valor. Esto puede ahorrarle tiempo y hacer que su código sea más conciso.

```
let num = 3; // TypeScript infers that x is of type number
console.log(num); // result: 3
```

Union type

Los 'union type' o tipos unión permiten especificar que una variable puede tener un tipo de entre varios tipos posibles (ojo solo uno de los tipos especificados, el tipo sigue siendo estático). Esto puede resultar útil cuando trabajamos con datos que podrían ser de más de un tipo.

```
let value: string | number;

value = 'hello';
console.log(value); // result: hello

value = 42;
console.log(value); // result: 42
```


Enums

Los [enums](#) son una forma de definir un conjunto de constantes con nombre . Hacen que su código sea más legible al darle nombres a valores que de otro modo serían difíciles de entender. Por otro eliminan fallos por ‘factor humano’ escribiendo incorrectamente.

```
enum MovieListType {  
    Popular,  
    Incoming,  
    MostRated,  
    NowPlaying  
}  
  
const movieListType = MovieListType.Popular;
```

```
enum MovieListType {  
    Popular = 'POPULAR',  
    Incoming = 'INCOMING',  
    MostRated = 'MOST_RATED',  
    NowPlaying = 'NOW_PLAYING',  
}
```

Type guards

Los 'type guards' permiten comprobar y limitar el tipo de una variable dentro de un bloque de código. Permiten escribir código que funcione con variables de diferentes tipos de forma segura.

```
function greeting(name: string | string[]): void {  
  if (typeof name === 'string') {  
    console.log(name.toUpperCase());  
  } else {  
    console.log(name.map(name => name.toUpperCase()).join(', '));  
  }  
}
```

```
greeting('John'); // result: JOHN  
greeting(['John', 'Doe']); // result: JOHN, DOE
```

Clases

Las [clases en TypeScript](#) son “similares” a las clases en otros lenguajes de programación orientados a objetos como Java y C# . Le permiten definir propiedades y métodos para un tipo particular de objeto y admiten herencia y encapsulación. TypeScript admite funciones de programación orientada a objetos como clases y herencia.

Las clases en TS se comportan parecido a JS pero seguimos teniendo algunas ventajas:

- **tipado estático** en propiedades y métodos.
- modificadores de acceso: **public**, **private** y **protected** (en lugar de # y _ respectivamente).
- **implementación de interfaces** como clases.

Generics

Los genéricos permiten escribir código reutilizable que funcione con una variedad de tipos.

Permiten crear funciones, clases e interfaces que pueden funcionar con cualquier tipo, en lugar de estar vinculados a un tipo específico.

```
function toArray<T>(arg: T): T[] {  
  return [arg];  
}  
  
let result: string[] = toArray<string>('hello');  
console.log(result); // result: ['hello']
```



Documentación

- [Web oficial.](#)
- [Referencia.](#)
- [Quick sheets.](#)



Ejercicio.

En este ejercicio vamos a crear un nuevo proyecto con vite incorporando Typescript como dependencia de npm para que pueda procesar archivos .ts de typescript. Una vez creado incorporaremos el código del ejercicio de la sesión 7 en el que realizamos peticiones http con fetch a la API Rest de TMDB. Ahora definiremos los tipos necesarios para trabajar con los datos de las películas.